Week 4 Status Report

PIC16F72 Microcontroller Simulator

Team: Mohammed Ayman, Mohammed Shayaan, Prakrithi, Sancia

Team and Responsibilities

Member	Role	Responsibility this week
Mohammed Ayman	Team Leader, Integration	Connecting the Java UI with the Python core, end-to-end testing, GitHub merge, documentation
Mohammed Shayaan	UI	Java Swing user interface [what changed this week]
Prakrithi	Core	CPU, instructions and execution in Python [what changed this week]
Sancia	Logging	Execution trace and logs [what changed this week]

Week 4 Objective

Enhance the existing simulator by separating it into **three independent processes** and enabling communication between them using an appropriate **POSIX IPC mechanism**.

Change of Programming Language (Core: Java to Python)

Decision: The simulator core (CPU, instructions, fetch/decode/execute, registers, memory) is now written in **Python**. The **user interface remains in Java (Swing)**. The project is now a two-language system. The Python core uses the Win32 API.

Component	Language	Owner
Core (CPU, instructions, execution)	Python with Win32 API (was Java)	Prakrithi

Component	Language	Owner
User Interface	Java Swing (unchanged)	Mohammed Shayaan
Logging / execution trace	[Python / Java]	Sancia
Integration (Java UI and Python core)	[Python + Java]	Mohammed Ayman

Original decision (Week 1): Java for the whole project, chosen for OOP modelling of CPU, memory and peripherals, team familiarity, and Swing for the UI.

Reason for the change: Java could not use POSIX IPC mechanisms, so the core was moved to Python using the Win32 API.

Advantages for this project:

- Python with the Win32 API can provide the IPC and synchronization the team wanted on Windows.

Limitations and risks:

- Two languages need an interface between them.
- Both Java and Python must be installed to run the simulator.
- The Win32 API only works on Windows, so the Python core is not portable to Linux or macOS.

Why Java was not kept for the core: Java could not use POSIX IPC mechanisms.

How the Java UI and Python core communicate: [e.g. TCP socket on localhost / pipes / subprocess stdin-stdout. Name the mechanism, the message format, and the file names on each side.]

Effect on development:

- Core classes and the 8 implemented instructions were rewritten in Python.
- The Java UI no longer runs CPU logic. It sends commands (Load, Reset, Step, Run) and displays what the core returns.
- Behaviour is unchanged: same instructions, same FETCH, DECODE, EXECUTE trace, same demo program results.

Completed Work

Area	Owner	Completed
Core	Prakrithi	pic_cpu_backend.py
UI	Mohammed Shayaan	SimulatorUINative.java
Logging	Sancia	pic_logger.py
Integration	Mohammed Ayman	Integration

Concurrency and Synchronization

Item	Details
Main thread	Listens on 127.0.0.1:45772 and starts one client thread for each Java UI connection.
Client threads (one per connection)	Read the LOAD, STEP, RESET and QUIT lines from the socket and put them on the commands queue.
Executor thread (one)	Takes commands from the queue one at a time, runs them on the CPU, sends the reply to the client, then updates shared memory.
Event worker thread (one)	Takes messages from the events queue, prints them, and sends them to the logging process on port 45773.
Semaphore (cpu.sem, Semaphore(1))	Works as a lock. Protects the CPU state (W, PC, memory, flags, halted, program) during step, load and reset, and while publish() copies the state.
commands queue (Queue, max 16)	Thread-safe producer-consumer queue. Client threads put, the executor gets. put() waits when the queue is full, so all commands run one at a time, in order.
events queue (Queue, max 16)	Same pattern for log events. event() uses put_nowait, so a full queue drops the event instead of blocking the CPU.

Item	Details
stop (threading.Event)	Shutdown flag set by QUIT and at exit. Every loop checks it, and queue reads time out every 0.2 s so threads notice it.
Shared memory	multiprocessing.shared_memory named pic16f72_sim_shared, 84 bytes: W, PC, Z, C, halted and memory 16-31. Only the executor thread writes it, after each command.
Inter-process communication	TCP sockets on localhost: port 45772 for the Java UI, port 45773 for the logging process.
Known limitation	The shared memory write has no cross-process lock, so a reader in another process could briefly see a half-updated state.
File(s)	pic_cpu_backend.py

Design Changes (and why)

- Core language changed from Java to Python (see the language change section above).

Individual Contributions

Member	Contribution
Mohammed Ayman	[Integration]
Mohammed Shayaan	[UI/UX]
Prakrithi	[Core]
Sancia	[Logging]